

Homework 2

CI/CD with Jenkins, GitHub, Docker & Docker Swarm on AWS

GitHub · Jenkins · Docker · Docker Swarm · Amazon ECR · Amazon EC2

Overview

This homework focuses on a complete, automated CI/CD pipeline: a change pushed to GitHub triggers Jenkins to build a Docker image, push it to a private Amazon ECR repository, and deploy it to a Docker Swarm cluster running on Amazon EC2 - with no manual steps in between.

Deliverables:

- A GitHub repository with the application source, Dockerfile, Jenkinsfile, and docker-stack.yml
- A private Amazon ECR repository holding the built, tagged images
- A Docker Swarm cluster on EC2 (at least 1 manager + 1 worker) running the deployed service

Format: Individual or team (instructor's choice per section). Total: **100 points** across 5 categories.

Learning Objectives

By the end of this assignment, students will be able to:

- **Version control:** manage source code on GitHub and configure a webhook that triggers CI automatically on push
- **CI/CD pipelines:** write a Jenkinsfile that builds, tags, pushes, and deploys a Docker image without manual steps
- **Containerization:** build a production-ready Docker image and run it locally before shipping it
- **Container registries:** authenticate Docker to Amazon ECR and push/pull private images by tag
- **Orchestration:** stand up a Docker Swarm cluster on EC2 and deploy a service that pulls its image from ECR
- **Cloud security:** configure IAM permissions and EC2 security groups so only the intended traffic and services can reach the pipeline and the cluster

Assignment Tasks

1. **GitHub repository** -- create a repo and push the application source, a Dockerfile, a Jenkinsfile, and a docker-stack.yml. Use feature branches and at least one pull request.
2. **Dockerize the application** -- write a Dockerfile, build the image locally, and confirm it runs and serves traffic on the expected port before touching AWS.
3. **Provision AWS resources** -- create a private Amazon ECR repository; launch EC2 instances (1 manager + at least 1 worker) and initialize Docker Swarm across them.
4. **Configure the Jenkins pipeline** -- write a Jenkinsfile with stages to checkout the code, build the image, authenticate to ECR (aws ecr get-login-password) and push a tagged image, then deploy by updating the stack on the Swarm manager.
5. **Trigger and verify** -- push a change to GitHub and confirm the pipeline runs automatically end to end: a new image lands in ECR, and the running service on the Swarm cluster updates to it.
6. **Document and submit** -- write the README and short report described below, then submit per the Submission Requirements.

Pipeline Flow

1. Developer pushes code to GitHub
2. GitHub webhook triggers the Jenkins pipeline
3. Jenkins builds and tags the Docker image
4. Jenkins authenticates to Amazon ECR and pushes the image
5. Jenkins tells the Swarm manager (EC2) to redeploy with the new image
6. Docker Swarm pulls the new image from ECR and rolls it out across the EC2 nodes

Submission Requirements

- **Repository link** -- GitHub repo containing the app, Dockerfile, Jenkinsfile, and docker-stack.yml, with full commit history
- **ECR repository** -- name/URI of the Amazon ECR repository and region it's in
- **Live deployment** -- public IP or domain where the deployed service is reachable, kept running through the grading window
- **Pipeline evidence** -- screenshot or console log of a successful Jenkins build (build, push, and deploy stages all green)
- **README** -- explains what the project is, how to run it, and how to verify the deployment
- **Short report (1-2 pages, PDF or Word)** -- architecture overview, screenshots of the ECR repo/Jenkins pipeline/running app, and challenges encountered
- **Due date:** [16-10-2026]
- **Submit via:** [Telegram group]

Late submissions, group-size limits, and academic-integrity policy follow the course syllabus.

Grading Scale

Category	Points
GitHub	15
Docker & Amazon ECR	20
Jenkins CI/CD Pipeline	30
Docker Swarm on EC2	25
Documentation	10
Total	100

Letter grade	Score
A	90-100
B	80-89
C	70-79

Letter grade	Score
D	60-69
F	below 60

GitHub -- 15 points

Criterion	Points	Full credit looks like
Repo structure	5	Clear layout (app source, Dockerfile, Jenkinsfile, docker-stack.yml), .gitignore excludes secrets/build artifacts
Commit history	5	Frequent, incremental commits with descriptive messages -- not one giant final commit
Webhook configured	5	A GitHub webhook triggers the Jenkins pipeline automatically on push (not manual “Build Now” only)

Docker & Amazon ECR -- 20 points

Criterion	Points	Full credit looks like
Dockerfile quality	5	Small, sensible base image, no unnecessary layers/files copied in, one clear process per container
Builds & runs locally	5	docker build succeeds and the container serves traffic on the expected port before any AWS step
ECR repository	5	A private Amazon ECR repository exists, correctly named, in a documented region
Push / pull from ECR	5	Jenkins authenticates with aws ecr get-login-password and pushes an image tagged with the build number or commit SHA -- not just latest

Jenkins CI/CD Pipeline -- 30 points

Criterion	Points	Full credit looks like
Pipeline as code	5	A Jenkinsfile checked into the repo (not a pipeline configured only by clicking in the UI)
Build stage	5	Automatically builds the Docker image on trigger
Push stage	5	Pushes the built image to ECR with a meaningful, traceable tag
Deploy stage	10	Automatically updates the running service/stack on the Swarm cluster to the newly pushed image
Trigger	5	Pipeline fires on a GitHub webhook or SCM poll, not only a manual build

Docker Swarm on EC2 -- 25 points

Criterion	Points	Full credit looks like
Cluster formed	5	At least 1 manager + 1 worker EC2 instance in the swarm (docker node ls shows all nodes Ready)
Stack deployed	5	docker stack deploy -c docker-stack.yml runs the service, pulling its image from ECR
Service reachable	5	The app is reachable on the expected port via the manager's or a node's public IP
Rolling update	5	restart_policy and update_config are set, and a Jenkins-triggered redeploy updates the service without dropping all replicas at once
Security groups	5	EC2 security groups allow only the required traffic -- the app port, SSH from a known source, and swarm's 2377/7946/4789 between nodes

Documentation -- 10 points

Criterion	Points	Full credit looks like
README	5	Explains what the project is, how to run it, and how to verify the deployment
Report	5	Short written report covering architecture, screenshots (ECR, Jenkins, running app), and challenges encountered

Total

Category	Points possible	Points earned
GitHub	15	
Docker & Amazon ECR	20	
Jenkins CI/CD Pipeline	30	
Docker Swarm on EC2	25	
Documentation	10	
Total	100	